AIStudio

Quickstart

Version: 1.2.2 | Updated: 2026-06-14

Get a running AIStudio instance in under 30 minutes.

AIStudio runs entirely on your Mac — no cloud account, no API keys, no data leaving your machine. You'll have a local AI search engine over your own documents, accessible from your browser.

> ***This guide is intentionally detailed*** *— it explains what each tool does and why it is needed. This is deliberate: AIStudio has several components, and understanding what you are installing makes you a much more effective user. If you simply want to get running as fast as possible, a streamlined install script is coming in a future release — but you will miss understanding how a RAG system is built.*

A Note on the Tools Used in This Guide

Installing AIStudio requires two external tools that work in a similar way — both are places where software is stored and from which your Mac can download and install it automatically:

Homebrew (https://brew.sh) is a package manager for macOS. Think of it as an App Store for developer tools — instead of clicking buttons in a GUI, you type `brew install <something>` and Homebrew finds it, downloads it, and installs it for you. We use it to install Python and Ollama.

GitHub (https://github.com) is where AIStudio's code lives. Think of it as a library where software is stored and versioned. We use a tool called `git` to download ("clone") AIStudio from GitHub directly onto your Mac. AIStudio is a public repository — no account or access key required to download it.

Both tools do the heavy lifting of finding, downloading, and installing software so you don't have to do it manually.

Before You Start

Open Terminal first. Press **⌘ Space** (that's the Command key and the Space bar at the same time), type **Terminal**, press **Enter**.

You'll see a window with a prompt that looks something like this:

```
yourusername@Mac ~ %
```

The prompt text varies by machine and depends on your name — don't worry about it.

AIStudio setup uses the shell to get installed and also to perform tasks the AIStudio User Interface (UI), which runs in your browser, doesn't cover. The commands all start with `ais_` — like `ais_start`, `ais_stop`, `ais_bench`. These are presented in Step 7.

> ***What is a shell?*** *Terminal gives you access to the shell — a text interface for running commands on your Mac.*

If you close the Terminal window at any point, open a new one: press **⌘ Space** (Command key + Space bar), type **Terminal**, press **Enter**. Then re-run the last command you were on and continue from there.

Prerequisites

You need a Mac with Apple Silicon (M1/M2/M3/M4). Everything else you need — Python, Homebrew, Qdrant, Ollama — is installed in the steps below.

> *Not sure if you have Apple Silicon? Click the **Apple menu** (on the top left part of the screen) → **About This Mac**. Look for "Chip: Apple M1" (or M2/M3/M4). Intel Macs have not been tested — results may vary.*

---

1. Install Homebrew

Homebrew is a package manager for macOS — it installs the software AIStudio needs.

First, check if it is already installed; in the Terminal, type:

```
brew --version
```

If you see a version number — Homebrew is already installed. → **Skip to Step 2.**

If you see `command not found`, install Homebrew. You will be asked for your Mac password — this is the same password you use to unlock your Mac. Type it and press Enter. Nothing will appear on screen as you type — that is normal:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.
```

The installer will show a list of directories it will create. Press **Enter** to continue.

When installation completes you will see `==> Installation successful!` Modern Macs set the Homebrew PATH automatically. Verify:

```
brew --version
```

If you see a version number — you are all set. On older Macs, if you see `command not found`, run these three lines:

```
echo >> ~/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

---

2. Install Python

First, check if Python is already installed; in the Terminal, type:

```
python3 --version
```

If you see a version number where the second part is 10 or higher — for example Python 3.10, 3.11, 3.12, 3.13, or 3.14 — Python is recent enough for AIStudio. → **Skip to Step 3.**

"3.10 or higher" means: look at the number after the first dot. If it is 10 or more, you are good.

If you see `command not found` — Python is not installed. Install it via Homebrew:

```
brew install python@3.13
```

This takes less than a minute. You'll see a stream of messages — ignore them. All is good when you're back at the `%` prompt.

You may also see notices like:

```
[notice] A new release of pip is available
```

Ignore these — they do not affect AIStudio.

Verify:

```
python3 --version
```

Expected: `Python 3.13.x` or higher.

> *AIStudio uses type syntax (`float | None`) that fails on Python 3.9. The macOS system Python is often 3.9 — that's why we check.*

---

3. Install Ollama

Ollama (https://ollama.com) is a local model runtime — it downloads, manages, and serves AI language models on your machine. Think of it as a local equivalent of the OpenAI API, running entirely on your hardware.

Check if already installed; in the Terminal, type:

```
ollama --version
```

If you see a version number — Ollama is installed. → **Skip to "Start Ollama" below.**

If Ollama is not installed:

```
brew install ollama
```

This takes less than a minute. You'll see a stream of messages — ignore them. All is good when you're back at the `%` prompt.

> *You may see a notification saying **"Background Items Added — ollama is an item that can run in the background."** Click to dismiss or ignore it. This means Ollama will start automatically when your Mac starts up — which is exactly what you want.*
>
> *You may also see technical notes about `OLLAMA_FLASH_ATTENTION` and background services — ignore them. Ollama is installed correctly.*

Start Ollama — run this once now. It also ensures Ollama starts automatically every time your Mac starts up:

```
brew services start ollama
```

Also install **Pango** — required for PDF report generation (benchmark reports):

```
brew install pango
```

> ***Why Pango?*** *AIStudio's benchmark runner generates PDF reports. Pango is a text rendering library that weasyprint (the PDF engine) depends on. It's a system library —* `brew install` *handles it in under a minute.*

Verify it is running:

```
ollama list
```

If you see the column headers (NAME, ID, SIZE, MODIFIED) — Ollama is running correctly. The list may be empty — that is normal. You will download models next.

If you see an error:

```
ollama serve
```

Run this in a separate terminal tab to start Ollama manually.

---

4. Pull Required Models

AIStudio uses two types of models that do completely different jobs.

nomic-embed-text is the indexing model. When you upload a document, this model reads it and converts every passage into a set of numbers that capture its meaning — called an "embedding." When you ask a question, it finds the passages whose meaning is closest to your question. It is small (274 MB), fast, and runs invisibly. You will never interact with it directly. Everyone needs this model.

The **language model** — Llama or Mistral — reads the relevant passages found by the indexing model and writes a human-language answer with citations. This is what most people think of as "AI."

AIStudio works with two excellent language models: **Llama** (by Meta) and **Mistral** (by Mistral AI). Both produce high-quality answers. Llama tends to be more thorough; Mistral more concise. If you have space on your drive, download both and try them side by side — you can switch between them in the AIStudio UI at any time.

Which model to download — based on your Mac's memory:

To check your memory: click the Apple menu (on the top left part of the screen) → **About This Mac** → look for "Memory."

- **8 GB** — download `mistral:7b`. `llama3.1:8b` may be slow.
- **16 GB** — download `llama3.1:8b` and `mistral:7b`. Try both.
- **32 GB or more** — download both. Consider `llama3.1:70b` only if you have 64 GB.
- **64 GB** — download `llama3.1:70b` for the best answer quality.

> ***Do not download `llama3.1:70b` with less than 64 GB RAM*** *— the download will work but the model will not fit in memory and your Mac will become unresponsive when you try to use it.*

Step 1 — Download the indexing model (required for everyone):

```
ollama pull nomic-embed-text
```

Step 2 — Download a language model. Run one or both depending on your RAM:

For 16 GB RAM (recommended default):

```
ollama pull llama3.1:8b
```

Alternative — slightly more concise answers, good on constrained hardware:

```
ollama pull mistral:7b
```

For 64 GB RAM only — highest answer quality:

```
ollama pull llama3.1:70b
```

> ***Download times*** *depend on your internet speed. To check yours, go to fast.com. On a 100 Mbps connection, expect about 6 minutes for `llama3.1:8b` and 5 minutes for `mistral:7b`. The time estimate shown during download may fluctuate — starting at hours, then dropping to minutes. The actual time depends on your connection speed.*

> ***Note:*** *If your models show a modified date of several weeks ago — that's fine. They don't expire.*

---

5. Install Qdrant

Qdrant (https://qdrant.tech) is the database that stores your document chunks. When AIStudio ingests a document, it splits it into overlapping passages — typically a few paragraphs each — and stores them as vectors in Qdrant. When you query, AIStudio searches across all chunks to find the most relevant passages.

> ***Why not Homebrew?*** *Qdrant is a Rust-based binary not available via Homebrew — install it directly.*

Check if already installed; in the Terminal, type:

```
qdrant --version
```

If you see a version number — → **Skip to Step 6.**

If not installed, create a home for Qdrant's data:

```
mkdir -p ~/qdrant_storage
```

This command returns no output — silence means success.

Download and install the binary:

```
curl -L https://github.com/qdrant/qdrant/releases/latest/download/qdrant-aarch64-apple-darw
```

Then move it into place, add it to your PATH, and verify:

```
mkdir -p ~/bin
mv qdrant ~/bin/qdrant
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
qdrant --version
```

These five commands run in sequence — you can run them one at a time or paste them all at once. They are safe to re-run.

Expected: `qdrant 1.17.0` (or newer). You will also see:

```
<jemalloc>: option background_thread currently supports pthread only
```

This warning is harmless — ignore it.

---

6. Clone AIStudio

AIStudio is a public repository — no GitHub account, SSH key, or access token required. You can download it directly with a single command.

First — check if git is installed; in the Terminal, type:

```
git --version
```

If you see `git version 2.x.x` or newer — → **proceed to the clone below.**

If you see `command not found`, or macOS shows a dialog asking to install developer tools — run:

```
xcode-select --install
```

A dialog will appear saying "The xcode-select command requires the command line developer tools. Would you like to install the tools now?" — click **Install**.

A **License Agreement** will appear — click **Agree** to continue.

A progress dialog will appear. The time estimate may start very high — even showing hours — then drop to minutes within seconds. Ignore the initial estimate. On a 100 Mbps connection expect about 8-10 minutes. The estimate will fluctuate during the download.

When done, verify:

```
git --version
```

Expected: `git version 2.x.x` or newer.

> *If no dialog appeared when you ran `git --version`, trigger the installation manually with `xcode-select --install`.*

Now create a folder for AIStudio and clone the repository — run all four commands in sequence:

```
mkdir -p ~/Developer
cd ~/Developer
```

```
git clone https://github.com/mbarberony/AIStudio.git
cd AIStudio
```

> ⚠️ ***Important:*** *Make sure you run* `mkdir -p ~/Developer` *and* `cd ~/Developer` *before cloning. AIStudio expects to live at* `~/Developer/AIStudio/` *— every command in this guide uses that exact path. If you accidentally cloned to* `~/AIStudio/` *instead, move it now:* `bash mkdir -p ~/Developer && mv ~/AIStudio ~/Developer/AIStudio`

This downloads about 96 MB — expect under 2 minutes on a 100 Mbps connection. When complete you will see `Resolving deltas: 100% (...), done.` and return to the prompt.

---

7. Install AIStudio Commands

`./ais_install` does two things: creates the Python environment and installs all dependencies, then makes the `ais_*` commands available in your Terminal. The commands all start with `ais_` — like `ais_start`, `ais_stop`, `ais_bench`. Here is how we make them available to you — the last step will let you see what they are and what they do.

```
cd ~/Developer/AIStudio
./ais_install
source ~/.zshrc
ais_help
```

This takes 2–3 minutes. You'll see progress messages as dependencies are installed.

> *You may see a series of* `WARNING: Cache entry deserialization failed` *messages — these are harmless pip cache warnings, not errors. The install will continue normally. Ignore them.*
>
> *You may also see a notice like* `[notice] A new release of pip is available` *— ignore it.*

You should see a list of all available AIStudio commands organized by category. Run `ais_help <command>` at any time for detailed help on a specific command.

> *You only need to run* `source ~/.zshrc` *once — right after installing. Every new Terminal window or tab you open in the future will load your commands automatically.*

---

8. Activate the Virtual Environment

Each time you open a new terminal tab — which should be rare after this session if you install a shortcut on your Desktop in Step 11 — activate the virtual environment before running AIStudio commands. A virtual environment is an isolated Python workspace that keeps AIStudio's dependencies separate from the rest of your system.

```
source ~/Developer/AIStudio/.venv/bin/activate
```

This command returns no output — silence means success. Your prompt will show `(.venv)` confirming the environment is active.

> *You do not need to activate the virtual environment manually before running `ais_start` — it handles this automatically. You only need to activate it manually if running `ais_*` commands from a fresh terminal tab before `ais_start` has run.*

---

9. Start AIStudio

```
ais_start
```

AIStudio starts three processes — Qdrant, Ollama, and the FastAPI backend — and opens the UI in your browser automatically. AIStudio starts in about 20 seconds.

> *You may see a warning that the backend was slow to start — this is normal on first run. The UI will still open correctly.*
>
> ***First run:** On a fresh install, `ais_start` automatically indexes the **demo** and **help** corpora in the background. You might see a progress banner in the UI while indexing completes — if indexing takes more than 30 seconds this banner will appear. Wait for it to complete before asking your first question.*
>
> ***First query:** The first query may take 20–50 seconds while the model loads fully into memory. Subsequent queries will be faster — typically 6–7 seconds.*

It is OK to **minimize** the Terminal window now — but **do not close it**. The Terminal is where the backend runs. Closing it will shut down AIStudio.

To stop AIStudio when you're done:

```
ais_stop
```

To verify the backend is up:

```
curl http://localhost:8000/health
```

Expected: `{"status": "ok"}`

---

10. What You're Looking At

When AIStudio opens in your browser, you'll see three main areas:

On the left — the corpus selector. This is where you choose which document collection to query. The **demo** corpus is already loaded — 9 original documents spanning 2003–2026, covering enterprise architecture, IT strategy, financial services, and agentic AI.

In the center — the chat area. Type a question, get a cited answer. References below each answer show exactly which document and page the answer came from. Click **Open** ↗ to see the source.

On the right — the settings sidebar. Controls how AIStudio retrieves and generates answers. See Step 12.

Try these questions on the demo corpus to start: - *"What is QFD and how does it apply to technology architecture?"* - *"How should a CTO prioritize a three-year technology strategy?"* - *"What are the key principles for modernizing legacy applications?"*

Then try switching to the **help** corpus and asking: *"How do I re-ingest a corpus?"* — AIStudio is answering questions about itself, using the same RAG pipeline to retrieve answers from its own documentation.

---

11. Add a Desktop Shortcut (Optional)

If you minimized the Terminal window in Step 9, reopen it now — press **⌘ Space** (Command key + Space bar), type **Terminal**, press **Enter**.

If you'd like to launch AIStudio by double-clicking an icon instead of opening a terminal:

```
ais_create_shortcut
```

This creates `AIStudio.app` in `~/Applications` and adds a shortcut to your Desktop. Double-clicking it starts all AIStudio services and opens the browser automatically — no Terminal needed.

> ***Icon not showing correctly?*** *Run `killall Dock` to refresh the icon cache.*

If, in the future, you want to remove the shortcut:

```
ais_create_shortcut --remove
```

To skip the Desktop symlink and only create the app in `~/Applications`:

```
ais_create_shortcut --nodesktop
```

AIStudio is ready. Your documents are waiting.

★★★ ★★★

For a full guided walkthrough — including the SEC 10-K at-scale exercise and benchmarking — see TUTORIAL.md.

---

12. Tuning Parameters

Parameter	Default	Effect
Top K	5	Chunks retrieved per query. Higher = more context, slightly slower. **Use 10 for the demo corpus and the SEC 10-K corpus** — the demo has small documents (as few as 20 chunks) that only surface reliably at K=10; the SEC corpus needs K=10 for cross-firm multi-source queries.
Temperature	0.3	LLM creativity. Lower = more factual. Keep at 0.3 for document Q&A.
Retrieval Mix	0.5	Blends keyword matching with semantic understanding. Drag left toward **Literal** (exact word matching) or right toward **Conceptual** (finds related meaning even when exact terms differ). Center (0.5) works well for most queries; try full Conceptual for thematic questions, center-to-Literal for specific entity or term lookups.
Score Threshold	0.2–0.5	Filters out retrieved chunks that scored too low to be useful. Set it too high and you starve genuinely relevant chunks — especially dense

Parameter	Default	Effect
		financial-filing text, which the indexing model under-scores — so answers turn hedged ("I don't have information about…"). Configured per-corpus — the demo uses 0.3; sec_10k uses 0.2 (the lower floor keeps the dense 10-K tables from being filtered out).

> *For more on query settings, see HOWTO.md.*

Troubleshooting

⌘K clears the terminal — when the terminal fills up with output, press **Command + K** to clear the screen. Your session and commands stay active. Useful after long ingests or benchmark runs.

Close and reopen Terminal if something unexpected happens. Press **⌘ Space** (Command key + Space bar), type **Terminal**, press **Enter**. Re-run the last command you were on and continue from there. A fresh terminal always starts with a clean environment.

`brew --version` **returns** `command not found` — Homebrew not installed. Run the installer in Step 1.

`python3` **not found** — run `brew install python@3.13`

`ollama list` **hangs** — Ollama is not running. If brew-installed: `brew services start ollama`. If .dmg installed: `ollama serve` in a separate tab.

UI shows "Ollama not running" — run `brew services start ollama` then `ais_start` again.

UI shows "Error loading corpora" on startup — the browser opened before the backend finished starting. Hard-refresh (`Cmd+Shift+R`). If it persists, run `ais_stop` then `ais_start`.

`(.venv)` **not in prompt** — run `source ~/Developer/AIStudio/.venv/bin/activate`

Stats show 0 chunks — corpus not yet ingested. Use the UI to add and ingest documents.

Qdrant not found — `~/bin` not in PATH. Run `source ~/.zshrc`.

Commands suddenly vanish (`zsh: command not found`) mid-session — your terminal's working directory was deleted or renamed while the shell was open (you'll also see `getcwd: cannot access parent directories`). Fix:

```
cd ~/Developer/AIStudio
source ~/.zshrc
```

This resets the working directory and reloads all aliases. Run `ais_start --version` to confirm.

`ais_*` command not found after install — run `source ~/.zshrc` to reload your aliases. **Important:** run it as a standalone command on its own line — do not chain it with `&&` or paste it together with other commands in the same block. A `source` inside a pasted sequence runs in a subshell and the aliases do not persist. After sourcing, verify with `ais_start --version`.

Tools Used in This Guide

Tool	What it does	URL
Homebrew	Package manager for macOS	https://brew.sh
Python	Programming language AIStudio runs on	https://www.python.org
Ollama	Local AI model runtime	https://ollama.com
Qdrant	Vector database for document storage	https://qdrant.tech
GitHub / git	Code hosting and version control	https://github.com

---

For architecture context, see docs/architecture_decisions.md. For day-to-day usage, see HOWTO.md. For guided walkthroughs, see TUTORIAL.md.